How the orders service serves a read

A read of one order, `GET /orders/42`, is answered from Redis when it can be and from Postgres when it cannot. This note shows the path, what a miss costs, and where the time of the slowest reads goes.

The path

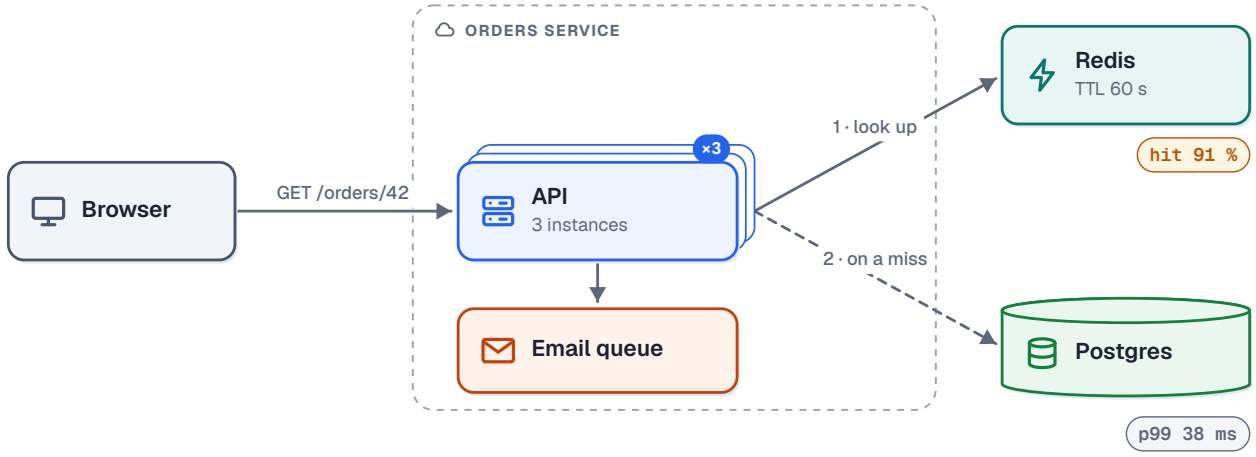


Figure 1: Three API instances behind one address. Redis answers first; Postgres only on a miss (the dashed arrow). Emails leave through a queue, so a slow mail provider never holds a request open.

> **Key idea**
> A miss is not an error, only a slower read: the API fills the cache on its way back, so the next read of the same order is a hit.

One read that misses

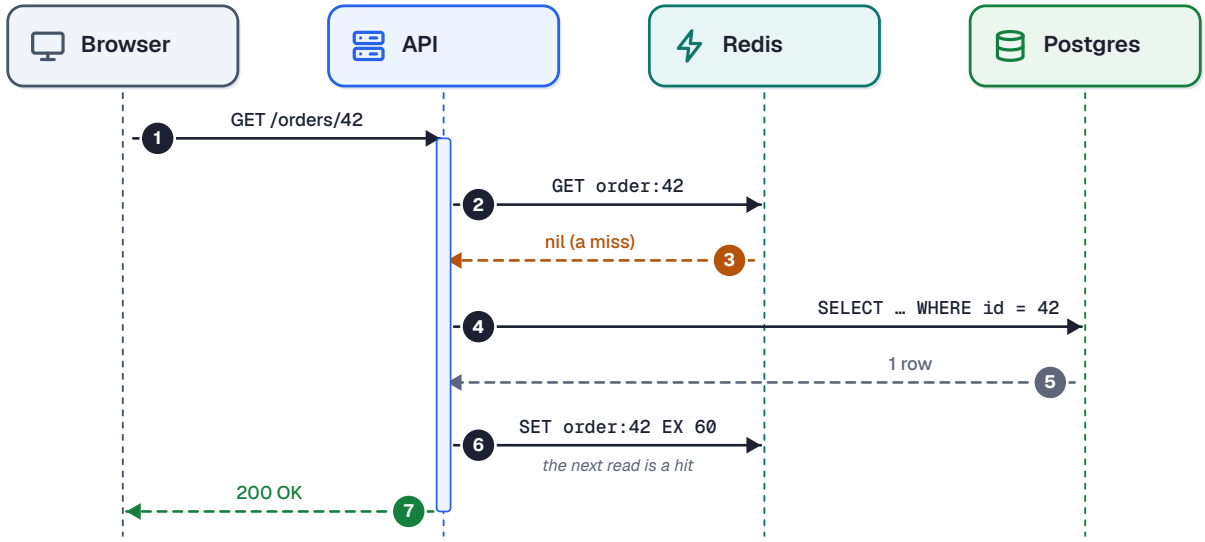


Figure 2: Seven messages for a miss, against three for a hit. Step 6 is what makes the next read fast.

What it costs

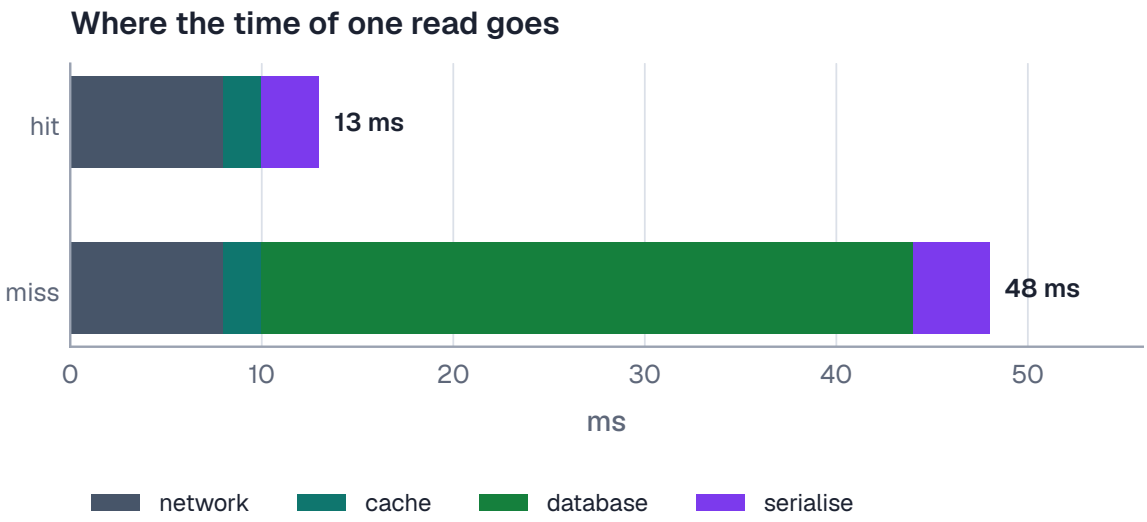


Figure 3: A hit spends its time on the network; a miss adds a database round trip that is two thirds of its total.

Endpoint	Reads / day	p50	p99
`GET /orders/{id}`	612 000	44 ms	212 ms
`GET /orders?customer=`	148 000	61 ms	180 ms
`GET /orders/{id}/lines`	96 000	39 ms	121 ms

Table 1: The three busiest reads last week.

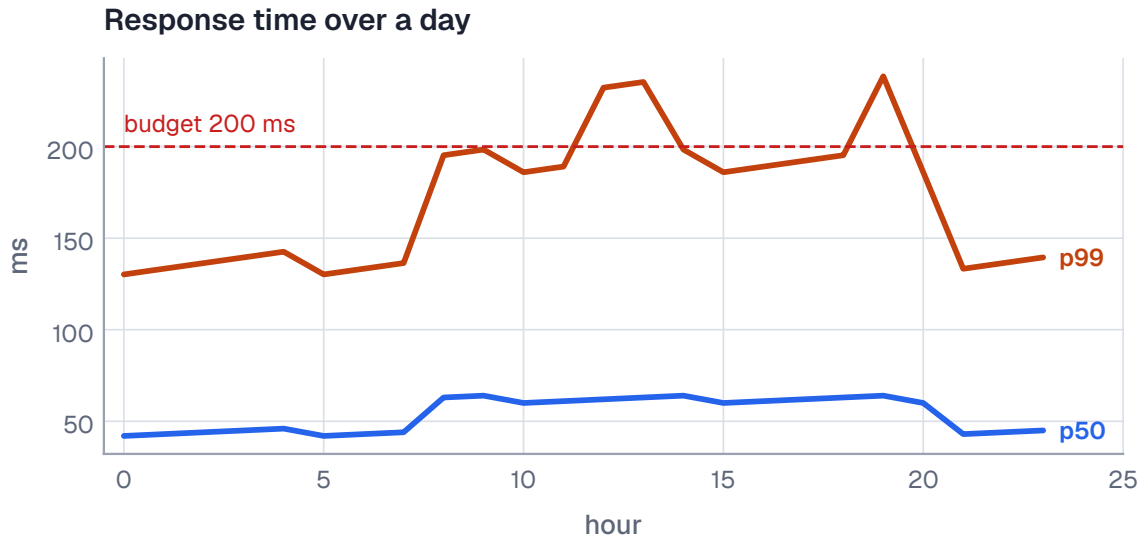


Figure 4: p99 crosses the 200 ms budget three times a day, each time when the cache has just been cold.

> **Watch**
> The hit rate fell from 94 % to 91 % this week. Each point lost moves about 6 000 reads a day onto Postgres.